Highlights

Procedural and learning-based generation of coral reef islands

- A hybrid sketch-based and learning-driven pipeline for coral reef island terrain generation
- Dual-view user sketches provide intuitive semantic and geometric control
- Procedural geological priors enable fully synthetic dataset generation
- A pix2pix conditional GAN learns realistic reef morphologies from labels
- Real-time generation of geologically plausible reef islands from sparse sketches

Procedural and learning-based generation of coral reef islands

Abstract

We propose a procedural method for generating single volcanic islands with coral reefs based on user sketches from two projections: a top view defining the island's shape, and a profile view, establishing its elevation. We add a user-defined wind field that deforms the island to mimic long-term effect of wind and wave, providing finer user control over the island's shape. Next, we model the coral growth on the island following biological observations. This results in a terrain height field of the island and its surrounding reef. Our method creates a large variety of coral reef island models which we used to train a conditional Generative Adversarial Network (cGAN). By applying data augmentation, the cGAN enhances the variety in the generated islands, providing users with greater freedom and intuitive controls over the shape and structure of the final output.

Keywords: Procedural terrain generation, Sketch-based modeling, Generative adversarial networks, Coral reef islands

1. Introduction

The scarcity of high-resolution data, the need for volumetric and multi-scale representations, and the strong influence of biological and hydrodynamic processes present unique modelling challenges for underwater landscapes. Among these environments, coral reef islands are of particular interest and complexity. They result from a long-term interplay of volcanic activity, coral growth, erosion, and subsidence, producing highly distinctive morphologies that procedural techniques must capture if they are to generate convincing seascapes. We propose a simplified user-centered interface to focus on the overall large-scale modelling of coral reef islands using sketching methods to avoid tedious user control over a large range of ecological parameters.

According to Charles Darwin's subsidence theory [1], these islands begin as volcanic landmasses as magma from the Earth's mantle erupts through the ocean floor and builds up layers of volcanic rock until rising above sea level (fig. 2).

In tropical waters, these volcanic islands create ideal conditions for coral reefs to develop. Initially formed as fringing reefs attached to the island's coastline (Stage 1 in fig. 3), reefs gradually morph into barrier reefs (Stage 2 in fig. 3) and atolls (Stage 3 in fig. 3) through the combination of long-term and short-term processes, notably subsidence (slow sinking process of the island), coral growth, and landmass and coral erosion processes [2].

The formation of these islands involves processes at multiple scales, from the growth patterns of coral colonies to large-scale sediment transport, which are difficult to simulate directly. As a result, purely procedural or physics-based simulations fail to produce convincing or diverse coral reef island landscapes. On the other hand, deep learning methods are inoperable due to the extremely small amount of data, and the scarcity of high-resolution DEMs of these regions.

Despite advances in terrain generation, current methods struggle to support user-controlled design of specific island shapes and achieving realism without real data. Coral reef islands exemplify this gap: we lack datasets to directly train deep learning models, and purely procedural methods require expert tuning to mimic their features.

To address the scarcity of real-world coral reef island data and the limitations of purely procedural approaches, our pipeline combines procedural generation with a conditional Generative Adversarial Network (cGAN). First, a procedural model uses algorithmic rules to generate a large and diverse set of synthetic island examples, each consisting of a terrain height field and a corresponding semantic label map that identifies regions such as ocean, reef, lagoon, beach, and mountain. Although procedural methods efficiently encode basic formation patterns, they are inherently rigid, restricted to a narrow range of shapes, and often rely on unintuitive parameters. These synthetic examples are therefore used to train a cGAN, which learns to generate island height fields conditioned on semantic label maps. Through exposure to numerous procedurally generated examples and data augmentation, the cGAN captures complex terrain characteristics and variations that extend beyond handcrafted procedural rules. Once trained, the cGAN becomes the primary generation tool: users can provide semantic maps through digital drawing or other simple methods, and the network generates plausible coral reef island terrains without requiring the procedural model. In this way, procedural generation serves as a means of creating training data, while the cGAN provides a flexible and expressive solution for the final terrain generation process.

The key contributions of this paper are:

- a novel sketch-based procedural algorithm for shaping island terrains from top and profile views,
- the use of a deep learning model trained on synthetic data derived from procedural rules, acting as an abstraction layer that hides underlying complexity,
- a demonstration that the cGAN approach tolerates imprecise, low-detail user input sketches, broadening usability, without the need for cutting-edge network architectures,
- and an insight that procedural generation remains essential

Figure 1: Given a free hand-drawn sketch of the different regions of an island (bottom inset of each example), our method generates a corresponding height field using neural networks. Subsidence (gradual sinking of land) is controlled by the user in the luminosity channel of the input. These examples show the gradual subsidence of the same island over time.

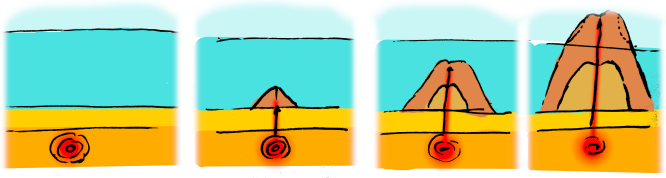


Figure 2: Volcanic islands rise above hotspots. The rise of magma from the hotspot forms a seamount. Consecutive eruptions from the volcano grow the seamount until the peak emerges above sea level. The ground above and close to sea level is affected by hydraulic, aeolian, and coastal erosion.

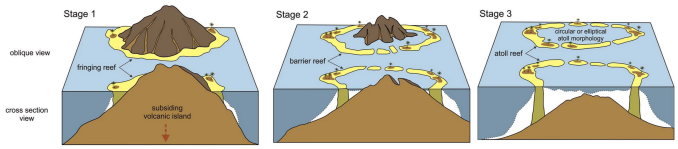


Figure 3: Coral colonies grow near sea level, forming a fringing reef (Stage 1). The island sinks slowly (subsidence); the coral reef continues its growth to keep living coral colonies in the photic zone. The sinking island increases the size of the lagoon, forming a barrier reef island (Stage 2). When the peak of the island is below the surface of the lagoon, an atoll is formed (Stage 3) [3].

to produce training data in data-sparse domains, illustrated by the example of coral reef islands.

These contributions collectively show a pathway for blending user-driven design with learning-based generation for terrain modelling.

2. Related Work

Terrain generation methods can be broadly classified into three families: procedural and simulation-based models, sketch-based control techniques, and learning-based synthesis. Our approach draws from all three, combining procedural control with learning-based realism.

Noise- and simulation-based terrains. Procedural noise functions such as Perlin and simplex noise, and fractal Brownian motion (fBm), have long been used to synthesize diverse terrains efficiently [4, 5, 6, 7, 8]. Although these methods provide scalability and interactivity, they lack explicit correspondence to geological or biological processes, limiting their realism for coral reef morphologies. Physical and simulation-based models incorporate erosion, uplift, or sediment transport to achieve higher realism [9, 10, 11, 12, 13, 14]. However, these simulations are computationally expensive and are typically calibrated for continental or fluvial terrains rather than shallow marine systems. As a result, neither class of approaches directly captures the coupled volcanic, subsidence, and coral growth phenomena characteristic of reef islands.

Sketch-based control. To increase artistic control and interactivity, sketch-based methods allow users to specify structural constraints or silhouettes directly [15, 16, 17, 18, 19, 20]. Curve and gradient-domain formulations enable intuitive authoring of shapes and elevation fields while maintaining spatial coherence. These approaches are particularly relevant for procedural workflows because they reduce parameter tuning and expose higher-level control to non-expert users. Our method extends this paradigm through a dual-view sketch interface, combining top and profile sketches, with stroke-defined deformation fields while preserving semantic labels that later condition the learning stage.

Learning-based terrains. Data-driven methods learn terrain appearance from examples, often through adversarial or diffusion-based generative models. Generative adversarial networks (GANs) and their conditional variants (cGANs) have been used to synthesize height fields and realistic landscapes [21, 22, 23, 24, 25, 26]. Among these, pix2pix [27] remains an effective baseline for translating semantic maps to terrain elevations. Recent diffusion models [28, 29, 30] achieve improved detail and realism but at significantly higher computational cost, making them less suitable for real-time interaction. In contrast, our approach relies on a lightweight pix2pix architecture trained entirely on synthetic data produced by the procedural sketch model, bridging controllable authoring and learned realism.

In summary, previous methods either offer control with limited realism (procedural/sketch-based) or realism without control (learning-based). Our work unifies both by procedurally encoding geological and biological priors into a dataset for a conditional generative model capable of real-time, user-driven synthesis of coral reef islands.

3. Description of our method

Our method for procedural generation of coral reef islands is composed of two independent modelling techniques shown in fig. 4. First, a curve-based algorithm (top-left blue block, presented in section 4) parametrises the surface of islands via a top-view sketch interface, describing the outlines of its constituent regions and a stroke-based wind field deforming them, as well as a profile-view sketch describing, for each region, its altitude and resistance to deformations. User inputs are given as 1D functions (altitude and resistance), a 1D polar function (island outlines), and a 2D vector field (wind field), as shown in fig. 5, which are convenient to randomise through the use of noise. While effective for encoding geological principles and generating structured examples, this algorithm cannot by itself provide the full freedom and irregularity required for realistic island design. This motivates the use of a learning-based component capable of generalising beyond the procedural rules.

Second, we propose a learning-based generation algorithm (right red blocks fig. 4, developed in section 6), which trains a neural generator G to transform a labelled image into a height field and a discriminator D to distinguish height fields provided from the training set against height fields generated by G. This adversarial training strengthens the outputs of the generator, up to the point where we discard the discriminator and training data, only keeping the generation step (bottom-right). Users are then able to provide unseen label maps and obtain height fields as close as possible to the training dataset. The procedural method lacks expressiveness but provides a large set of varied synthetic data; the cGAN offers expressiveness but requires a large dataset. Their combination is therefore natural: the procedural model acts as a data generator, while the cGAN removes its structural biases.

We connect these two algorithms through a process of dataset generation (central orange block fig. 4, described in section 5) in order to enforce the benefits of each while reducing their limitations. Given the initial curve-based user inputs, we create a dataset composed of similar samples processed by our first algorithm, rasterised into a 2D image of the parametrised regions, and paired with the resulting height field. We then significantly augment the dataset by employing various data augmentation techniques: translations, scaling, and copy-pasting of multiple islands in a single sample. We then obtain a dataset large enough for training our cGAN and enable users to procedurally create coral reef islands with a high level of control.

4. Procedural island generation

We propose a structured process for generating coral reef island terrains that takes the user's sketches and produces a complete 3D terrain model. This process begins with the creation of an initial height field based on user inputs, and applies wind deformation to introduce natural variations, and finally integrate coral reef features via subsidence and coral growth modelling.

4.1. Initial island

A top-view sketch defines the island's outline as seen from above. Using a simple drawing interface, the user can delineate the boundaries between key regions of the island, including the island itself, beaches, lagoons, and surrounding abysses. This system assumes that these regions are arranged concentrically around the centre of the island, with each boundary defined by a radial distance from the centre.

Each region's boundary is represented in polar coordinates $\left(r_{\mathbf{p}}, \theta_{\mathbf{p}}\right)$, with $r_{\mathbf{p}}$ indicating the radial distance from the island's centre and $\theta_{\mathbf{p}}$ representing the angular position. This polar representation allows us to map the user's sketch onto a circular framework, ensuring smooth transitions between regions and maintaining a coherent island layout.

In this sketch, the user defines the overall horizontal layout of the island, including the size and shape of each feature. Variations in the outline are introduced by allowing the radial distances to vary with angle, ensuring that the island is not strictly symmetrical and introducing more natural, irregular shapes.

A profile-view sketch defines the vertical elevation profile of the island along any radial direction, offering control over the island's height. In this view, the user specifies the elevation of different regions of the island, such as the island peak, beach, lagoon, abyss, and everything in between, by drawing the corresponding profile curve.

These regions correspond to key terrain transitions: the highest point of the island (centre), the island border, the beach, the lagoon, and the deep-sea abyss. We interpolate these milestones into a continuous 1D height function $h_{\text{profile}}(\tilde{x}_{\mathbf{p}})$, where $\tilde{x}_{\mathbf{p}}$ represents a non-uniform region distance from the island's centre to the point $\mathbf{p}$, such that $h(\mathbf{p}) = h_{\text{profile}}(\tilde{x}_{\mathbf{p}})$ gives the height at each point. The profile is continuous at the order of the interpolation method; results of this work are obtain using piece-wise linear interpolation, resulting in a C^0 continuous surface.

By combining the top-view and profile-view sketches, our method generates a coherent 3D terrain model that accurately reflects the user's design by revolution modelling.

The result is a height field that captures both the radial structure of the island (from the top-view sketch) and the vertical elevation profile (from the profile-view sketch), producing an organic representation of islands with smooth transitions between the key terrain features. Three slightly edited height functions on an identical island layout and their associated outputs are presented in fig. 6. To avoid perfectly smooth surfaces, we also apply a very low-amplitude Perlin noise perturbation to the final height field, adding fine-scale irregularities.

Instead, we approximate the morphological outcome of erosion with a deformation field: user-drawn strokes define a wind velocity field that warps the island, introducing large-scale asymmetries without explicitly removing matter. To account for the fact that not all regions respond equally to erosion, we also introduce a resistance function, which modulates deformation's strength across regions. For example, shallow coastal areas are strongly deformed, while the deep abyss or resistant volcanic core remain largely unaffected. This combination provides an efficient and intuitive proxy for the long-term shaping role of

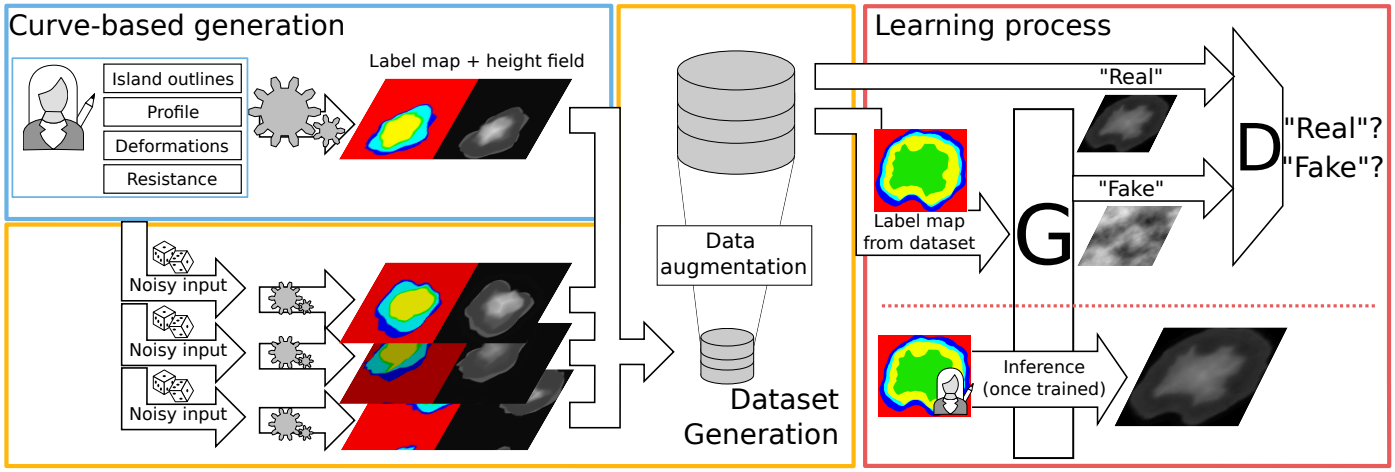


Figure 4: Blue box: our pipeline first prompts the user to create single pairs of coral reef island height fields and label maps using our proposed curve-based modelling algorithm (section 4). Yellow region: the same algorithm is applied multiple times on randomly altered versions of the user input to create a dataset, which we further enhance with data augmentation techniques (section 5). Redbox: finally, we train a conditional Generative Adversarial Network, which can then be used standalone to create height fields from labelled sketches (section 6).

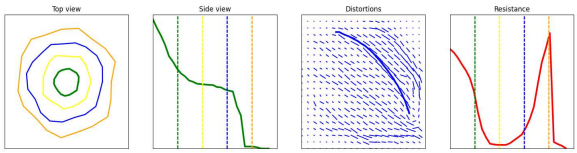


Figure 5: The user interacts directly on the island by editing the different canvases in any order. This UI shows, from left to right: the top-view sketch with the different outlines of each region, the profile-view sketch with the outlines represented by dotted lines, the wind velocity sketch drawn with strokes (last stroke is visible), and the resistance function showing here a high resistance at the top of the island and on the front reef.

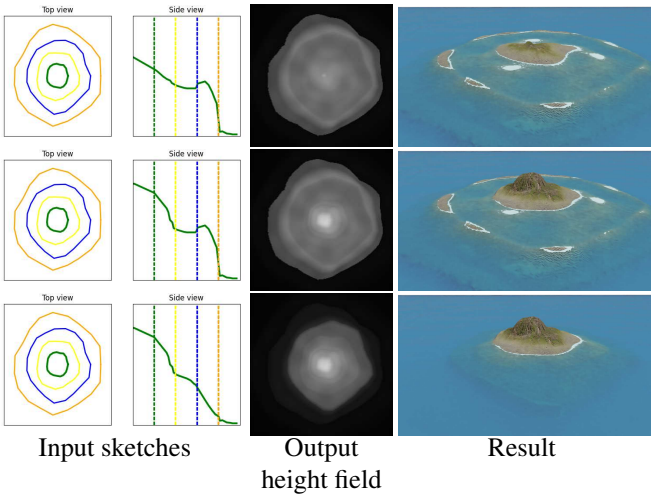


Figure 6: Providing a smooth function between each region results in islands with plausible reliefs. We fixed the outlines (first column) while editing only the height function (second column) in order to produce, from top to bottom, a low island, a coral reef island, and finally an identical island without the reef.

wind and waves. Although real coastlines are shaped by both wind and wave processes, we represent them jointly through a single user-defined wind field, which serves as a controllable proxy for their combined morphological effect.

The wind field is represented as a series of strokes drawn by the user on a 2D canvas. Each stroke represents a parametric curve, where the direction and strength of the wind are encoded as a vector field. The user controls the wind's direction by drawing a set of curves, and we interpret them to create a deformation field that defines how the terrain is warped.

The final deformation vector $\Phi(\mathbf{p})$ is computed as a sum of the influences from all strokes:

$$\Phi(\mathbf{p}) = \mathbf{p} + \sum_{C \in \text{curves}} \Phi_C(\mathbf{p}). \tag{1}$$

This process introduces variations in the terrain, distorting the coastline, creating concave regions, and breaking the original radial symmetry defined by the top-view and profile-view sketches.

To ensure that certain regions of the terrain such as deep-water areas remain relatively unaffected by the wind, a resistance function $\rho(\tilde{x}_\mathbf{p})$ is applied. The resistance function modulates the effect of deformation based on the previously computed piecewise parametric distance $\tilde{x}_\mathbf{p}$, with the same interaction means as the h_{profile} function.

The deformation vector previously described is then scaled by the resistance function at each point $\mathbf{p}$, such that the final deformation vector becomes

$$\tilde{\Phi}(\mathbf{p}) = \left(1 - \rho(\tilde{x}_\mathbf{p})\right) \cdot \Phi(\mathbf{p}). \tag{2}$$

Once the deformation vector $\tilde{\Phi}(\mathbf{p})$ is computed, the terrain height at point $\mathbf{p}$ is adjusted by displacing $\mathbf{p}$ to a new point $\tilde{\Phi}(\mathbf{p})$. We then compute the final height

$$h(\tilde{\Phi}(\mathbf{p})) = h_{\text{profile}}(\tilde{x}_\mathbf{p}). \tag{3}$$

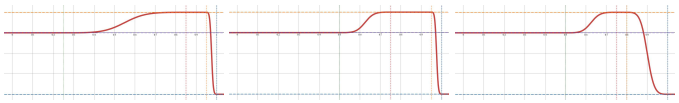


Figure 7: The modelling of the reef growth in our model is described by a piecewise function h_{coral} which is flat in the lagoon, the crest and abyss, and follows a smoothstep function as transitions for the back reef and fore reef regions. The model is easily edited by tuning the height values and delimitations of the subregions. Here, three examples of reef growth function with different delimitations (vertical dotted lines), providing more or less smooth transitions between reef subregions.

4.2. Coral reef modelling

To account for the island subsidence due to tectonic activity we propose to scale the initial height field downward, modelling procedurally the effect of the volcanic island slowly sinking into the ocean. The user provides a subsidence rate $\lambda \in [0, 1]$, which represents the proportion by which the island has sunk. We suppose the abyssal floor at $h = 0$.

As the volcanic island subsides, coral reefs grow upward to remain close to the water surface, following the "keep-up" strategy observed in most real-world coral formations. Coral growth is restricted to regions where the depth is within an optimal range for coral development, typically from the water surface to around 30 metres below before becoming much scarcer.

The coral reef features (reef crest, back reef, and fore reef) are modelled separately from the subsidence process. We generate a coral feature height field $h_{\text{coral}}(\mathbf{p})$, which remains unaffected by the island's subsidence.

In our model, coral reef growth is entirely independent of the subsided terrain. Even as the volcanic island sinks, coral growth is driven only by the proximity of terrain to the water surface, ensuring that coral features always remain near the surface, irrespective of how much the island subsides.

We define distinct reef zones anchored at specific heights, relative to water level. For instance, $h_{\text{crest}} = -2\,\text{m}$ (reef crest near sea level), $h_{\text{back}} = -20\,\text{m}$ (back reef and lagoon), and $h_{\text{abyss}} = -100\,\text{m}$ (fore reef sloping to abyss)

Each reef subregion is defined over a parametric domain $x \in [0, 1]$, with $x = 0$ the beginning of the reef region and $x = 1$ its end. For instance:

- Back reef: $x_{\text{back,start}} = 0$, $x_{\text{back,end}} = 0.5$,
- Reef crest: $x_{\text{crest,start}} = 0.75$, $x_{\text{crest,end}} = 0.8$,
- Abyss begins at $x_{\text{abyss,start}} = 1$.

We model transition zones between these regions using a smoothstep operator [31].

The final step is to blend the subsided height field $h_{\text{subsid}}(\mathbf{p})$ with the coral feature height field $h_{\text{coral}}(\mathbf{p})$ to produce the final terrain. To achieve this, we use a smooth maximum function defined as :

$$\text{smax}(a, b) = \begin{cases} a + \dfrac{1}{k} & \text{if } a = b, \\ a + \dfrac{b - a}{1 - e^{k(a-b)}} & \text{otherwise,} \end{cases} \tag{4}$$

which smoothly blends the two height fields, ensuring that the coral regions dominates where coral growth is present, while the subsided island terrain dominates in other regions. This blending method ensures that transitions between the coral and subsided regions are smooth and visually consistent.

Here, $a = h_{\text{subsid}}(\mathbf{p})$ is the height from the subsided island, $b = h_{\text{coral}}(\mathbf{p})$ is the height from the coral reef feature, and k controls the smoothness of the transition (fixed arbitrarily here at $k = 5$ for heights ranging between 0 and 1), guaranteeing a continuous and smooth surface. Higher values of k bring the smax function closer to the max function.

The resulting terrain represents a plausible coral reef island, where the volcanic island has subsided, and coral reefs have grown upward to keep pace with the water level.

One of the key strengths of this method is its flexibility, as the subsidence and coral reef growth processes are modelled independently, allowing for a wide range of configurations. Users can generate plausible island terrains with or without coral features, or apply the coral reef growth modelling to existing height fields from other sources.

5. Data generation

The creation of the dataset is done through the use of the procedural algorithm for which we alter the input parameters.

For each generation, the top-view and profile-view sketches use the user-given input. Each outline of the top-view sketch r is modified by adding continuous noise η, giving the final outlines:

$$r^*(\theta) = r(\theta) + \eta_{\text{outlines}}(\theta).$$

On the other hand, we define a new initial profile-view sketch by defining $h^*_{\text{profile}}(\tilde{x}_{\mathbf{p}})$, the initial height function on which fBm noise η_{height} is applied to obtain

$$h^*_{\text{profile}}(\tilde{x}_{\mathbf{p}}) = h_{\text{profile}}(\tilde{x}_{\mathbf{p}}) \cdot \eta_{\text{height}}(\tilde{x}_{\mathbf{p}}).$$

An identical process is done for the resistance function:

$$\rho^*(\tilde{x}_{\mathbf{p}}) = \rho(\tilde{x}_{\mathbf{p}}) \cdot \eta_{\text{resistance}}(\tilde{x}_{\mathbf{p}}).$$

Next, we generate a random deformation field. We do not target realism in stroke generation. Instead we generate a random number n of strokes with random paths. Spread and intensity of each stroke is also randomised, providing complexity and variety in the results. Our random wind stroke paths are smooth, simplex noise-driven trajectories biased toward the island's center, ensuring that distortions remain mainly around the main landmass.

Finally, to further enrich the dataset with erosion-like structures, we add a set of radial strokes of small width and low strength, extending between the island centre and the exterior of the canvas, oriented alternately landward and seaward. This produces simple radial drainage patterns illustrated in fig. 8, available by the radial definition of our islands contours.

Once all inputs are set, we generate an example for multiple levels of subsidence $\lambda \in [0, 1]$ producing height fields that incorporate coral reef modelling along with its associated label map.

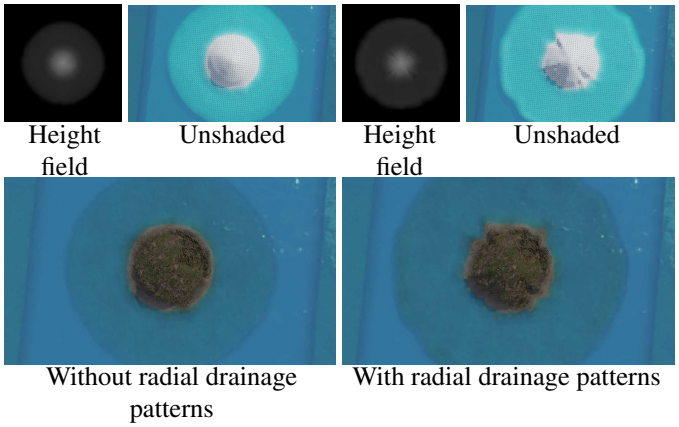


Figure 8: Multiple procedural strokes are added between the island centre and the surrounding sea, alternately landward and seaward. This transforms a simple circular island (left) into a more realistic volcanic island (right) by cheaply simulating star-shaped radial drainage patterns.

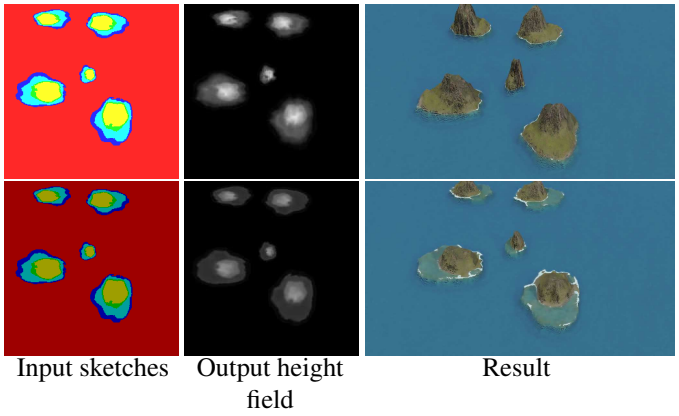


Figure 9: An identical label map yields similar height fields over multiple inferences from the model, even after modifying the subsidence factor (visible in the luminosity of the input image).

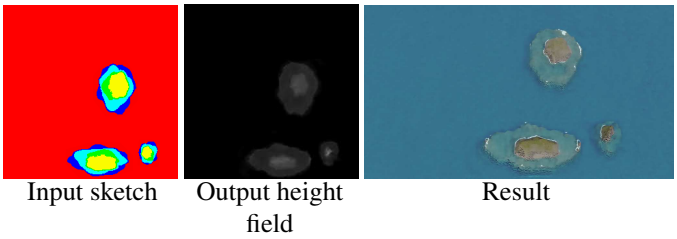


Figure 10: A sample of the dataset with the input and output of the island copied and scaled at multiple positions of the image, creating three different islands in a single image.

For the purpose of training the pix2pix model with HSV images as input, we use the Hue component to encode the labels from the label map. We introduced a new dimension to the model's learning capabilities by incorporating the subsidence rate, denoted as λ, into the Value component.

Moreover, we purposefully left the Saturation component unchanged at this stage, reserving space for potentially including another parameter in the future, which would allow us to expand the model's utility without altering the foundational HSV encoding scheme established during its initial training. By adhering to this encoding format, we ensure continuity in data representation, which maximises the efficiency of the pretrained model. This strategic update enhances the model's adaptability and broadens its applicability to tackle new, complex challenges more effectively.

This configuration enables rapid generation of large quantity of samples, with multiple parameters, of a single island centred in the image. We enhance each sample with several data augmentation techniques in addition to the usual affine transformations (rotation, scaling, and flipping).

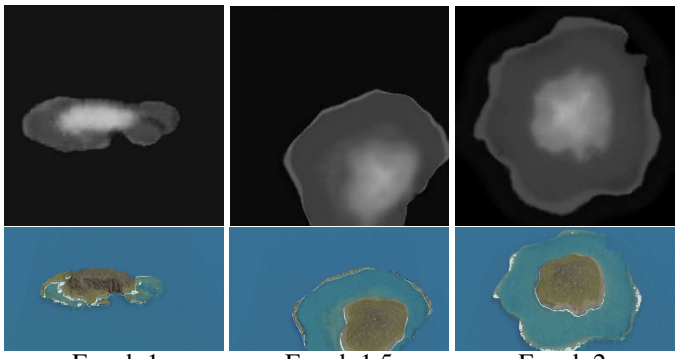
Translation: Since the original algorithm always centres the island, we translate the islands within the image to remove this constraint (fig. 10). This ensures that the cGAN can generate islands in any position within the frame. By wrapping the island on the borders, the model learns that data can appear on the edges of the image.

Copy-paste: We combine multiple islands into a single sample with only a maximum intersection over union (IoU) of 10%, allowing the abysses to overlap but without obtaining other region types to merge. Height fields are blend using the smooth maximum function from eq. (4), and label blending uses the usual max operator. Regions not covered by any island are assigned the abyss ID, which correspond to the minimal height.

All augmentation techniques are simultaneously applied to both the height field and the label map ensuring consistency between input (the label map) and output (the height field).

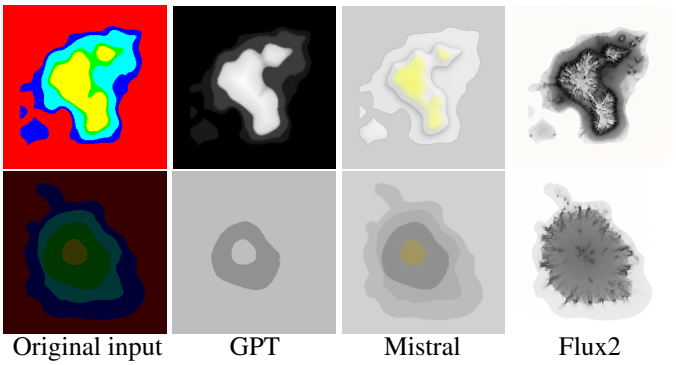
6. Learning-based generation

In this section, we introduce the use of a cGAN, specifically the pix2pix model, to enhance the island generation process by increasing the variety and flexibility of terrains. While the initial procedural algorithm generates diverse island examples (convex and concave outlines), cGAN provides additional flexi-



Epoch 1 (~10 000 samples) | Epoch 1.5 (~15 000 samples) | Epoch 2 (~20 000 samples)

Figure 11: After the first epoch (left), grid artefacts similar to a low-resolution image are visible, but are being corrected during a second epoch (middle) where erosion patterns appear in the height fields (right).



Original input | GPT | Mistral | Flux2

Figure 12: Results of GPT5.5, Mistral, and Flux2. Prompt: "Transform the segmented image of this island into a grayscale heightmap of the island. Red is open ocean, dark blue is coral reef, light blue is lagoon, green is beach and yellow is the island center. The darkness of the image represents the age of the island, meaning that darker images creates more eroded and subsided islands compared to light images. The image must not be shaded. Each pixel of the output represent the height of the terrain."

bility in generating more complex terrain without the rigid constraints of the procedural algorithm that stem from our initial assumptions based on coral reef formation theory.

Our model is trained using a batch size of 1, and optimised using the Adam optimiser with a learning rate of 0.0002 and momentum parameters $\beta_1 = 0.5$ and $\beta_2 = 0.999$. The loss function combined a conditional adversarial loss and an L1 loss, where the L1 loss was weighted by a factor $\lambda = 100$. The generator follows a U-Net architecture with encoder-decoder layers and skip connections, while the discriminator was based on a PatchGAN, classifying local image patches. These parameters are directly adopted from the original pix2pix paper [27]. We use a dataset of 10 000 samples that are all seen once during an epoch (representing about 30 minutes of training). 75% of the dataset is reserved for training, 12.5% for validation and 12.5% for testing sets.

During inference after a single epoch, our model still outputs grid artefacts, visible in fig. 11. After the second epoch, visual artefacts are already rare. The training time is relatively short compared to typical deep-learning terrain generation pipelines, largely thanks to the synthetic nature and consistency of our dataset.

Once the model is trained, the user colours a 256×256 RGB image to sketch the different regions. The subsidence factor is controllable through the luminosity of the input image. The inference time of our deep learning model remains constant regardless of scene complexity. Using the NVIDIA GeForce GTX 1650 Ti GPU with Python 3.10 and PyTorch version 2.5.1+cu121, the inference time measured is 5 ms (std 1.1 ms).

The output is a complete 2D height map representing the terrain's elevation at each point. Although the cGAN's internal logic is not easily interpretable, the label map remains available after inference and retains semantic terrain structure for the user.

Other deep learning architectures. We note that the the pix2pix original paper is more than a decade old, thus a comparison with modern models is necessary. However, the interactive condition (results in less than one second) eliminates most modern models. Secondly, the pix2pix model VRAM footprint is light (<4GB during training), while most models exceeds the capacity of low-end PCs (e.g.: pix2pixHD for 512×512 images is not usable on 4GB GPUs). Finally, we require the model to be flexible on the user inputs, meaning that the label map colors at any pixel may not exactly match a semantic map (see fuzzy inputs in fig. 13). For this last reason, models such as GauGAN requiring strict semantic inputs are unfit.

For illustration purpose, we proposed a simple prompt to three diffusion models inside the authors proposed interactive tools: OpenAI (GPT 5.5), Anthropic (Mistral Medium 3.5), and Flux2 (using the HuggingFace interface). We did not work on the prompt writting, it is possible that better results could be achieved with more work into it. Inference times using the proprietarian servers exceed greatly the interactive time limit (>6s for Flux2, >30s for Mistral Medium 3.5, and >45s for GPT 5.5). Results are shown in fig. 12.

7. Results

Allterrains presented in this work are defined in normalized coordinates $(x, y) \in [-1, 1]^2$ and $z \in [0, 1]$. Noise amplitudes used are bounded by $|\eta_{\text{outlines}}| \leq 0.2$, $0.8 \leq \eta_{\text{height}} \leq 1.2$, $0.8 \leq \eta_{\text{resistance}} \leq 1.2$ with 2 to 6 random wind strokes (with uniform sampling of spread $0.1 \leq \sigma \leq 0.3$ and strength $0.05 \leq S_C \leq 0.3$).

The resulting model for coral reef island generation enables a high level of user control as unconstrained painting allows complex scenarios while producing height fields in real-time. In this paper we used Blender to provide renders directly from the output height fields. As our pix2pix model is trained to output 256×256 images, the resolution of the 3D models is limited by this architecture.

By using deep-learning-based models, most initial constraints are removed (radial layout, isolated islands, ...). The control over the overall shapes of the island regions is given

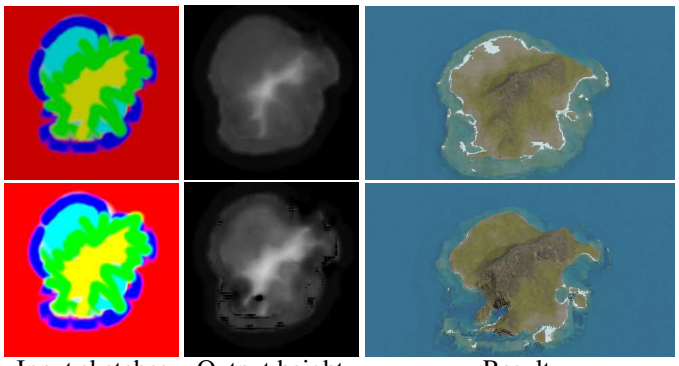


Figure 13: User applied fuzzy brushes to draw the label map, resulting in some pixels that are inconsistent with the dataset and illogical island layouts: abyss regions (red) are found between beach (green), lagoon (cyan) and reef regions (blue). The neural model ignores the layout inconsistencies, and partial overbrightness as long as the pixel is not completely white.

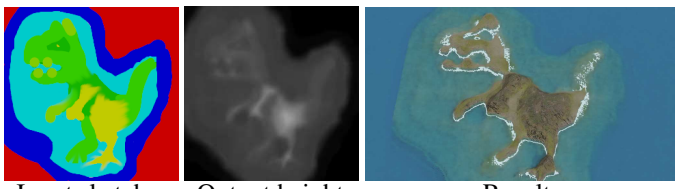


Figure 14: Without constraints on the generation, the user may use unrealistic layout and the neural network will however output a plausible result. Here new colours have been added (pistachio), but the network naturally considers it as a transition from mountain to beach.

through digital painting with any software, here using GIMP (first column of fig. 13) and Paint (such as in fig. 14). Each pixel of the image is encoded in HSV, with the region identifier encoded in the Hue channel. The user may increase or decrease the subsidence level of the island by modifying the Value channel over the whole image (see fig. 9).

Since the model relies on pixel-level statistics rather than strict class values, it is robust to noise caused by compression, anti-aliasing from brush tools, or resizing artefacts introduced by image editors. The example displayed in fig. 13 (top) displays a sketch with blurry outlines and unexpected layouts (see the small red regions inside the southern lagoon region or the adjacency of beach regions directly with the abyssal region). The learned model does not include inconsistencies and results in plausible 3D models. We can note that the input label map is not required to be hand-drawn, as a simple Perlin noise can produce it randomly, as shown in the examples of fig. 19. fig. 13 (bottom) shows highlight clipping (white pixels due to artificial oversaturation, losing the Hue value) that is not interpretable by the model and resulting in black patches in the result.

The tolerance to imprecise input values enables more control about the transitions between two regions than the procedural module. fig. 14 shows an example of input map with regions that are leaking over neighbouring regions, and the introduction of new hue values nonexistent in the dataset (light green and dark green) but are the interpolated hue values of mountain regions and beach regions.

Because our curve-based procedural phase included low randomness, the output of the cGAN is limiting its unpredictability, meaning that small input changes result only in minor changes on the output, preventing unexpected results. fig. 9 shows the result of an input map with only a variation on the subsidence level, the resulting height fields are very similar. Adding the real-time computation of outputs, it becomes possible to construct progressively a landscape and correct small inconsistencies to intuitively design islands inspired by real-world regions. A creation of an island similar to Mayotte, presented in fig. 15 was hand-made in just a few minutes.

fig. 16 (bottom) presents results of a second cGAN training for which the procedural parameters used for the generation of the training dataset "volcanic" are slightly adjusted for lower overall resistance and to have a crater in the center of islands (fig. 16, top). The h_{profile} function include a hole at the center of the island to emulate a volcaninc crater, which we can see strongly in the first and last examples, and more slightly in the second example; the resistance function ρ is lowered on the reefs, creating visible dents in the reef and dendritic patterns on the island sides.

8. Conclusion

In this work we presented a novel approach to generating coral reef island terrains by combining traditional procedural methods with deep learning techniques. We first developed a procedural generation algorithm capable of creating a wide variety of island terrains through a combination of top-view and

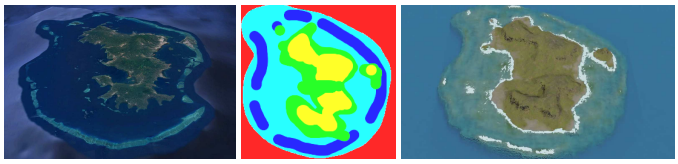


Figure 15: Comparison between real (left, from Google Earth) and synthetic (right) islands of Mayotte. The label map (centre) is hand-drawn to match approximately its real-world counterpart.

profile-view sketches, wind deformation, and subsidence and coral reef growth modelling. By applying these methods, we produced plausible terrains based on geological processes, capturing key features of coral reef islands: island, beaches, lagoons, and coral reefs.

To further enhance flexibility and realism in the generation process, we incorporated a Conditional Generative Adversarial Network, using the pix2pix model to generate height maps from label maps of island features. The cGAN model allowed us to overcome some of the constraints inherent in the procedural algorithm, such as radial symmetry and fixed island positioning. Through data augmentation, we trained the cGAN on a synthetic dataset, enabling the generation of varied and plausible island terrains.

One of the main strengths of this approach is its ability to produce a wide variety of island terrains, even in the absence of real-world data. The procedural generation methods allow for high flexibility in designing both the shape and features of the island, while the use of cGAN enables further refinement and the generation of terrains not bound by the original constraints of the procedural model. By combining these two methods, we leverage the complementary advantages of both: the structured control of procedural techniques and the pattern-learning capabilities of deep learning.

A key advantage of this approach is the preservation of semantic information throughout the generation process. The label map, which serves as the input to the cGAN, can also be used after terrain generation to provide a detailed semantic representation of the different regions of the island (island, beach, lagoon, coral reef, and abysses). This label map can be useful to guide post-processing operations, such as applying different textures based on terrain features or adding other environmental elements, vegetation for instance. The preservation of semantic information provides a useful connection to the next stage of terrain manipulation, making the process more versatile and adaptable to different use cases.

Furthermore, the use of an out-of-the-box cGAN model highlights the feasibility of employing existing neural network architectures with minimal modifications in the field of procedural generation. This is particularly important for domains where real-world data is scarce, such as coral reef islands, allowing synthetic data to be effectively used for training purposes.

While the cGAN model provides increased flexibility and variety in island generation, it does come with certain limitations:

Data-driven dependence. Just as any deep learning method, the quality of generated terrains strongly depends on the quality of the dataset. Unusual layouts or out-of-distribution inputs are not entirely interpretable by the models. Our dataset generation method lifts some restrictions, but some remain such as the required presence of all regions in the input map. This limitation reduces the true diversity of the generated terrains, as the cGAN cannot generate terrains that deviate too far from the examples in the training set.

Biases from the synthetic dataset. Since the cGAN model is trained entirely on a procedurally generated dataset, it inherits the biases present in the initial algorithm. For example, while the model breaks free from the radial symmetry constraint and centre positioning, it remains dependant on the structure and patterns of the synthetic data (e.g., our procedural drainage patterns, our reef analytic morphology, etc.). The ”realistic” aspect of the results thus depends on the generation methods used to create the dataset.

Lack of user control. Another limitation of using cGAN in this context is the lack of real-time user control during terrain generation. While traditional procedural generation methods allow users to adjust parameters during the generation process, the cGAN model operation is abstracted from the user, providing no mechanism for direct interaction beyond the initial label map. This reduces the level of customisation available to the user.

There are several directions for future research and improvements. One promising avenue is to incorporate the wind velocity field directly into the cGAN training process, potentially as an additional input condition. This would allow the model to better capture wind-driven terrain features such as cliffs or other deformations influenced by wind patterns.

Another area for exploration is improving user interaction during the terrain generation process. While the current model allows for rapid terrain generation, adding more options for users to interact with the cGAN, such as adjusting parameters (wind strength and main direction, erosion, reef function parameters, ...), could improve control over the terrain generation process of our system and help the user to generate his target results.

Finally, further improvements could be made to the synthetic dataset. Incorporating additional geological processes, such as higher fidelity physical simulation of wave and rain erosion, and tidal influences, could lead to even more realistic terrains at the cost of interactivity. Additionally, refining the way islands are blended in multi-island samples, or adding more diverse input conditions (e.g., different geological settings), could help the model generalise and produce more varied and dynamic landscapes.

Various other neural network models could be exploited to increase user interactions and improve user control, with newer variants of cGANs [32], or models with style transfer functionalities [33, 34] in order to change the overall aspect of a terrain [35, 36], use text-to-image models [37, 38] to generate height fields from a verbal prompt, or super-resolution models [39] to increase the definition of details in the final output [40].

References

[1] C. Darwin, C. Jackson, The Structure and Distribution of Coral Reefs: Being the First Part of the Geology of the Voyage of the 'Beagle,' under the Command of Capt. Fitzroy, R. N., during the Years 1832 to 1836, Vol. 12, 1842, p. 115. doi:10.2307/1797986.

[2] D. Hopley, Encyclopedia of modern coral reefs : structure, form and process, Springer, Credo Reference, 2014.

[3] J. P. Terry, J. Goff, One hundred and thirty years since darwin: 'reshaping' the theory of atoll formation, The Holocene 23 (2013) 615–619. doi:10.1177/0959683612463101.

[4] K. Perlin, An image synthesizer, ACM SIGGRAPH Computer Graphics 19 (1985) 287–296. doi:10.1145/325165.325247.

[5] K. Perlin, Noise hardware, Real-Time Shading SIGGRAPH Course Notes (2001).

[6] F. K. Musgrave, C. E. Kolb, R. S. Mace, The synthesis and rendering of eroded fractal terrains, Proceedings of the 16th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH 1989 (1989) 41–50doi:10.1145/74333.74337.

[7] D. S. Ebert, D. Peachey, S. Worley, J. C. Hart, K. Musgrave, K. Perlin, W. R. Mark, Texturing and Modeling, A Procedural Approach, Vol. Third edition, Morgan Kaufmann, 2003.

[8] E. Reinhard, A. Lagae, S. Lefebvre, R. Cook, T. DeRose, G. Drettakis, D. Ebert, J. Lewis, K. Perlin, M. Zwicker, State of the art in procedural noise functions, in: H. Hauser, E. Reinhard (Eds.), Eurographics 2010 - State of the Art Reports, The Eurographics Association, 2010. doi:https://doi.org/10.2312/egst.20101059.

[9] B. Beneš, V. Těšínský, J. Hornyš, S. K. Bhatia, Hydraulic erosion, Computer Animation and Virtual Worlds 17 (2006) 99–108. doi:10.1002/cav.77.

[10] X. Mei, P. Decaudin, B. G. Hu, Fast hydraulic erosion simulation and visualization on gpu, Proceedings - Pacific Conference on Computer Graphics and Applications (2007) 47–56doi:10.1109/PG.2007.27.

[11] B. Neidhold, M. Wacker, O. Deussen, Interactive physically based fluid and erosion simulation, Natural Phenomena (2005) 25–32.

[12] G. Cordonnier, J. Braun, M.-P. Cani, B. Benes, Éric Galin, A. Peytavie, Éric Guérin, Large scale terrain generation from tectonic uplift and fluvial erosion, Computer Graphics Forum 35 (2016) 165–175. doi:10.1111/cgf.12820.

[13] G. Cordonnier, M.-P. Cani, B. Beneš, J. Braun, Éric Galin, Sculpting mountains: Interactive terrain modeling based on subsurface geology, IEEE Transactions on Visualization and Computer Graphics 24 (2017). doi:10.1109/TVCG.2017.2689022.

[14] H. Schott, A. Paris, L. Fournier, E. Guérin, E. Galin, Large-scale terrain authoring through interactive erosion simulation, ACM Transactions on Graphics 42 (2023) 1–15. doi:10.1145/3592787.

[15] J. Gain, P. Marais, W. Straßer, Terrain sketching, Proceedings of I3D 2009: The 2009 ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games 1 (2009) 31–38. doi:10.1145/1507149.1507155.

[16] H. Hnaidi, E. Guérin, S. Akkouche, A. Peytavie, E. Galin, Feature based terrain generation using diffusion equation, Computer Graphics Forum 29 (2010) 2179–2186. doi:10.1111/j.1467-8659.2010.01806.x.

[17] F. P. Tasse, A. Emilien, M.-P. Cani, S. Hahmann, A. Bernhardt, First Person Sketch-based Terrain Editing, 2014.

[18] C. Gasch, M. Chover, I. Remolar, C. Rebollo, Procedural modelling of terrains with constraints, Multimedia Tools and Applications 79 (2020) 31125–31146. doi:10.1007/s11042-020-09476-3.

[19] F. X. Talgorn, F. Belhadj, Real-time sketch-based terrain generation, ACM International Conference Proceeding Series (2018) 13–18doi:10.1145/3208159.3208184.

[20] E. Guérin, A. Peytavie, S. Masnou, J. Digne, B. Sauvage, J. Gain, E. Galin, Gradient terrain authoring, Computer Graphics Forum 41 (2022) 85–95. doi:10.1111/cgf.14460.

[21] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial networks, Advances in Neural Information Processing Systems 3 (2014) 139–144. doi:10.48550/arXiv.1406.2661.

[22] A. Wulff-Jensen, N. N. Rant, T. N. Møller, J. A. Billeskov, Deep Convolutional Generative Adversarial Network for Procedural 3D Landscape Generation Based on DEM, Springer, 2018, pp. 85–94. doi:10.1007/978-3-319-76908-0˙9.

[23] R. Spick, J. Walker, Realistic and textured terrain generation using gans, in: European Conference on Visual Media Production, Vol. 2022-March, ACM, 2019, pp. 1–10. doi:10.1145/3359998.3369407.

[24] Éric Guérin, J. Digne, Éric Galin, A. Peytavie, C. Wolf, B. Beneš, B. Martinez, Interactive example-based terrain authoring with conditional generative adversarial networks, ACM Transactions on Graphics 36 (2017). doi:10.1145/3130800.3130804.

[25] E. Panagiotou, E. Charou, Procedural 3d terrain generation using generative adversarial networks (10 2020).

[26] C. Beckham, C. Pal, A step towards procedural terrain generation with gans (7 2017).

[27] P. Isola, J.-Y. Zhu, T. Zhou, A. A. Efros, Image-to-image translation with conditional adversarial networks, in: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, 2017, pp. 5967–5976. doi:10.1109/CVPR.2017.632.

[28] J. Ho, A. Jain, P. Abbeel, Denoising diffusion probabilistic models, Advances in Neural Information Processing Systems 33 (2020) 6840–6851.

[29] A. Nichol, P. Dhariwal, Improved denoising diffusion probabilistic models, Tech. rep. (2021).

[30] J. Lochner, J. Gain, S. Perche, A. Peytavie, E. Galin, E. Guérin, Interactive authoring of terrain using diffusion models, Computer Graphics Forum 42 (10 2023). doi:10.1111/cgf.14941.

[31] K. Perlin, Improving noise, in: Proceedings of the 29th annual conference on Computer graphics and interactive techniques, ACM, 2002, pp. 681–682. doi:10.1145/566570.566636.

[32] T. Park, M.-Y. Liu, T.-C. Wang, J.-Y. Zhu, Gaugan, in: ACM SIGGRAPH 2019 Real-Time Live!, Vol. 405, ACM, 2019, pp. 1–1. doi:10.1145/3306305.3332370.

[33] L. A. Gatys, A. S. Ecker, M. Bethge, A neural algorithm of artistic style (8 2015).

[34] D. Zhu, X. Cheng, F. Zhang, X. Yao, Y. Gao, Y. Liu, Spatial interpolation using conditional generative adversarial neural networks, International Journal of Geographical Information Science 34 (2020) 735–758. doi:10.1080/13658816.2019.1599122.

[35] S. Perche, A. Peytavie, B. Benes, E. Galin, E. Guérin, Authoring terrains with spatialised style, Computer Graphics Forum 42 (10 2023). doi:10.1111/cgf.14936.

[36] S. Perche, A. Peytavie, B. Benes, E. Galin, E. Guérin, Styledem: a versatile model for authoring terrains (4 2023).

[37] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer, High-resolution image synthesis with latent diffusion models (12 2021).

[38] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, I. Sutskever, Learning transferable visual models from natural language supervision (2 2021).

[39] C. Dong, C. C. Loy, K. He, X. Tang, Image super-resolution using deep convolutional networks (12 2014).

[40] Éric Guérin, J. Digne, Éric Galin, A. Peytavie, Sparse representation of terrains for procedural modeling, Computer Graphics Forum 35 (2016) 177–187. doi:10.1111/cgf.12821.

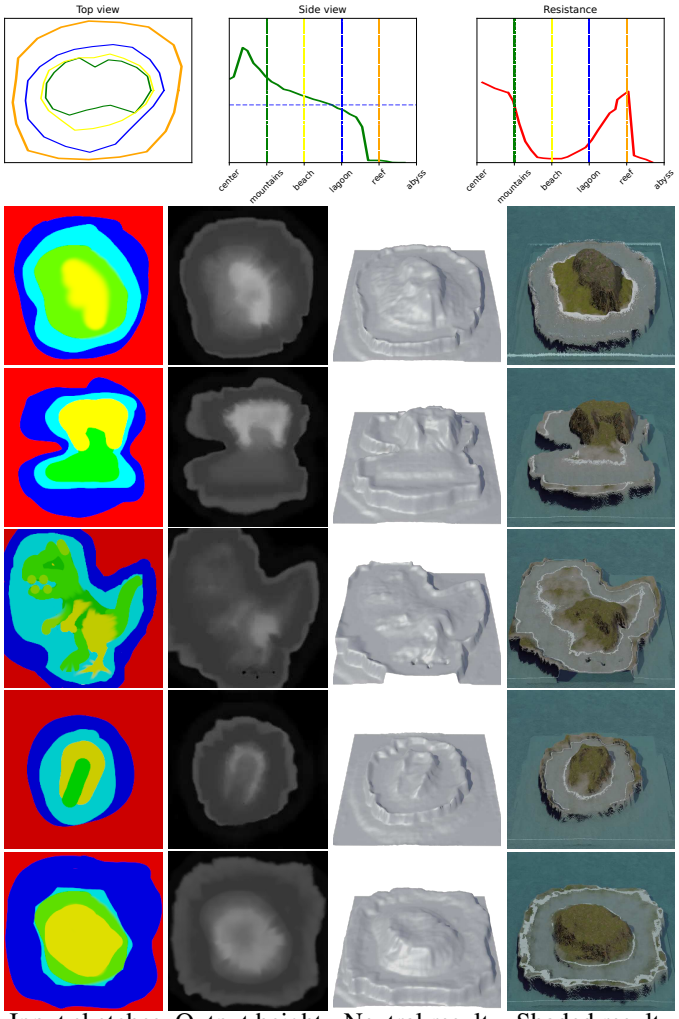


Figure 16: Top row: user input used to generate the "volcanic" dataset. Bottom: User sketches creates various islands, following the initial user inputs.

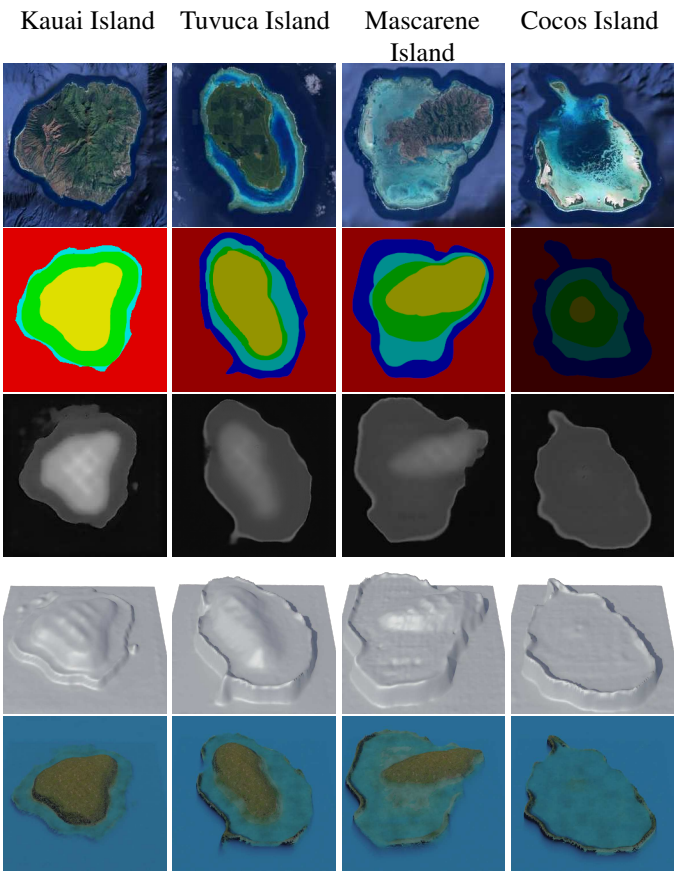


Figure 17: Reproduction of islands. From left to right: Kauai Island, Tuvuca Island, Mascarene Island, and Cocos Island.

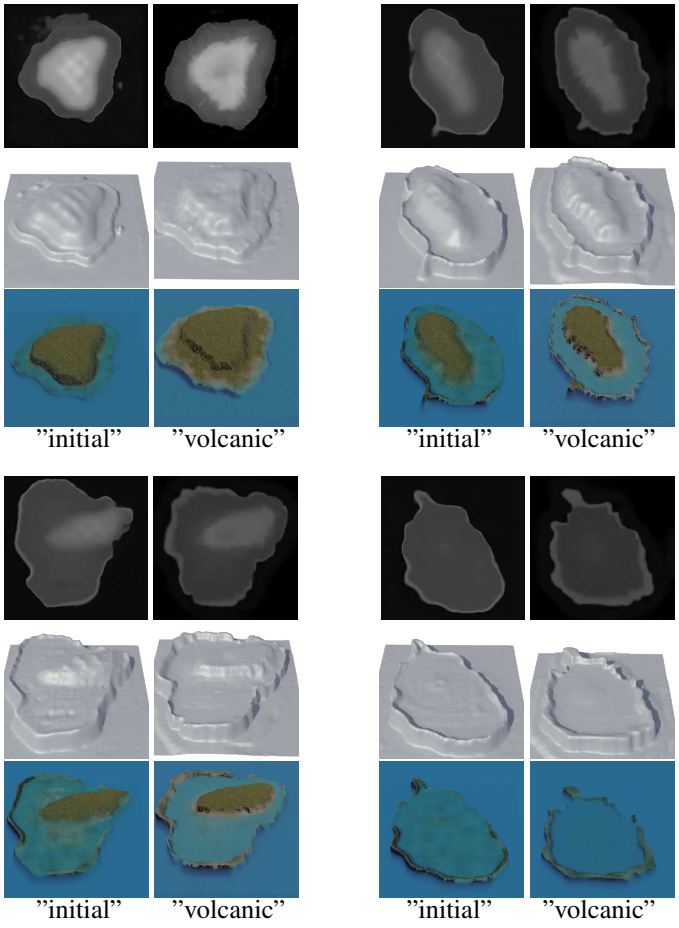


Figure 18: Comparison between the ”initial” cGAN trained model (dataset generated from parameters visible in fig. 5) and the ”volcanic” cGAN (fig. 16, top). The input label maps are identical as fig. 17.

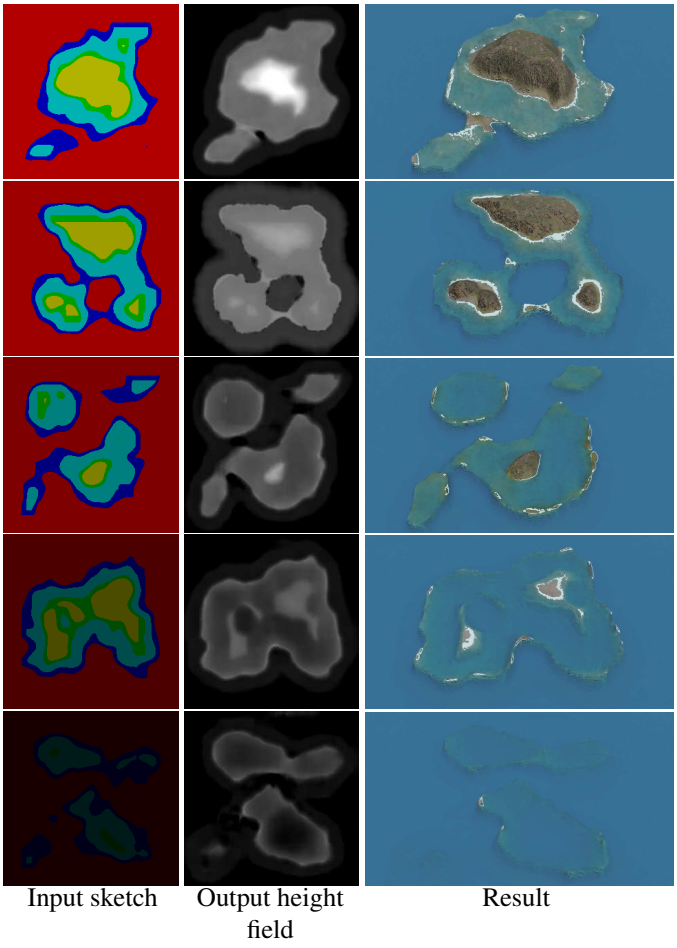


Figure 19: Starting from random Perlin noise, transformed into a label map, we can generate a large variety of results.

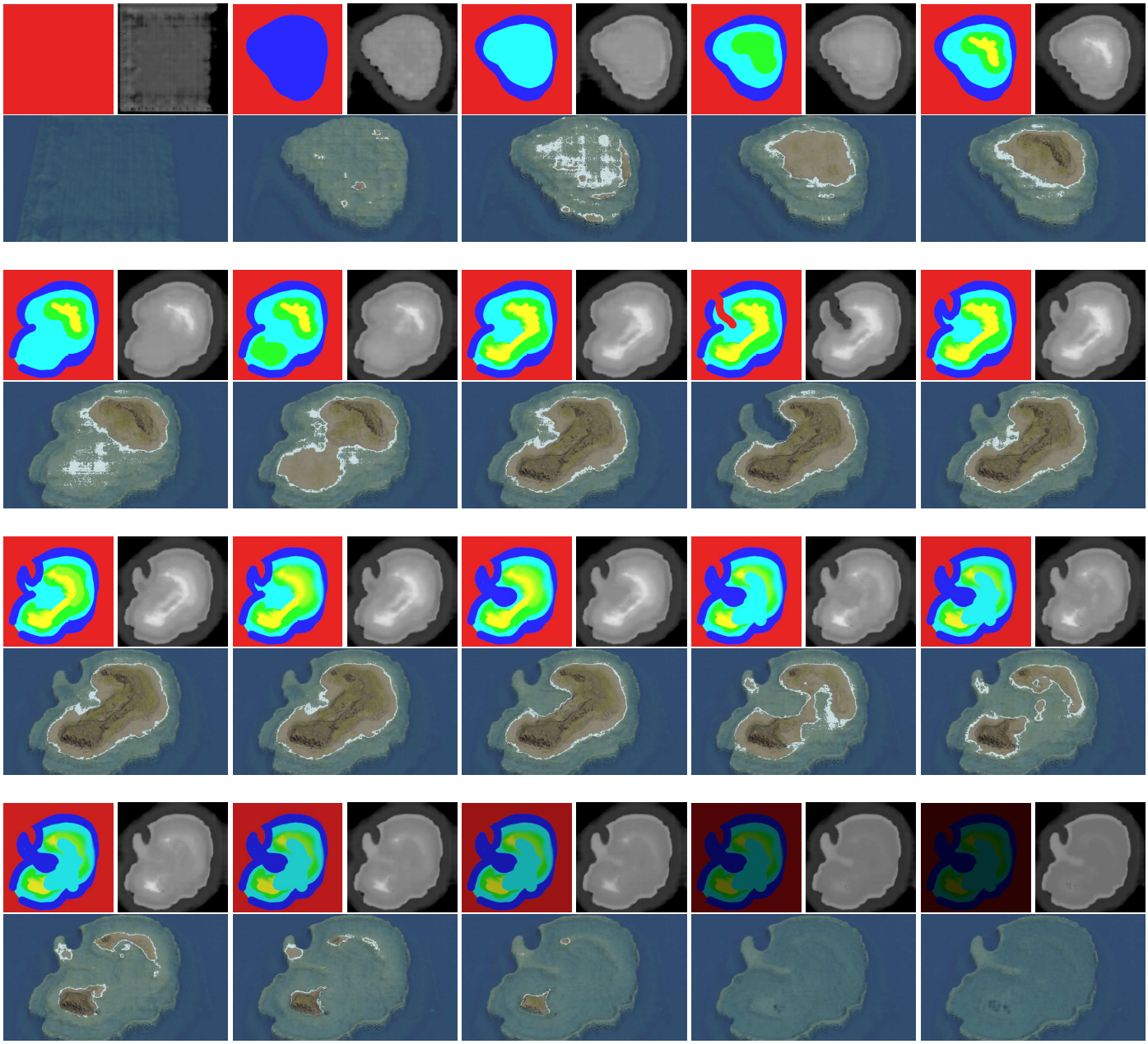


Figure 20: Few frames of an editing session for the generation of a coral reef island on Paint. The resulting height fields are output in real-time and each generation is consistant, allowing to interact fluently with the system without unexpected behaviour. The shaded outputs (bottom of each frame) are rendered in Blender, which are not real-time.